INSTRUKCJA LABORATORYJNA 4	Programowanie obiektowe (Java)
	Temat: **Odśmiecanie pamięci, referencje, pakiety, adnotacje**
	Przygotował: **mgr inż. Michał Młodawski**

CEL ZAJĘĆ

Po tych zajęciach powinieneś:

- wyjaśnić, jak JVM automatycznie zarządza pamięcią oraz jak garbage collector wyszukuje obiekty do usunięcia,
- znać kolektory dostępne w JDK 25 i wiedzieć, dlaczego `System.gc()` jest tylko prośbą do maszyny wirtualnej,
- rozróżniać referencje silne, miękkie, słabe i phantom oraz dobrać właściwą do zadania,
- zwalniać zasoby za pomocą try-with-resources i klasy `Cleaner` zamiast przestarzałej metody `finalize()`,
- grupować klasy w pakiety i poprawnie je importować,
- stosować wbudowane adnotacje Javy i zbudować prostą aplikację konsolową z biblioteką Picocli.

1. Zarządzanie pamięcią

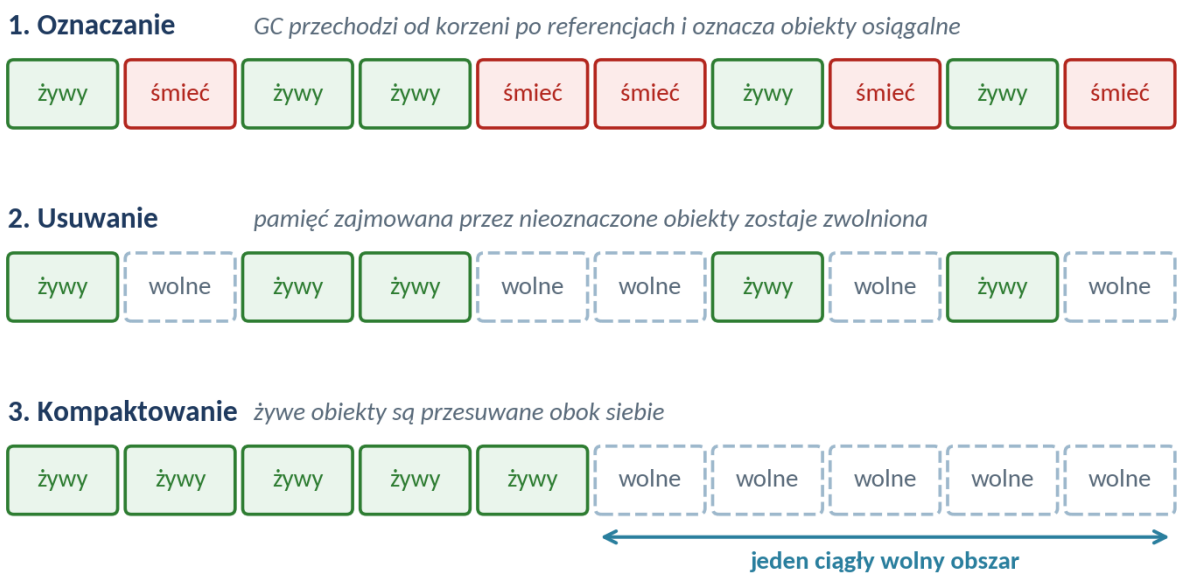
Wyobraźmy sobie, że piszemy rozbudowaną aplikację, która tworzy tysiące obiektów, ale w danej chwili korzysta tylko z części z nich. Skąd maszyna wirtualna Javy (JVM, czyli program, który wykonuje skompilowany kod bajtowy i zapewnia mu środowisko uruchomieniowe) ma wiedzieć, którą pamięć może odzyskać i wykorzystać ponownie, a której nie może ruszać, żeby nie stracić danych zapisanych w naszych zmiennych?

1.1. Strategie zarządzania pamięcią

JVM stosuje **automatyczne zarządzanie pamięcią**. Pamięć dla obiektu jest przydzielana na stercie za każdym razem, gdy program tworzy go operatorem `new`, a zwalniana, gdy obiekt przestaje być potrzebny. Programista nie zwalnia pamięci ręcznie, tak jak robi się to w C funkcją `free()` albo w C++ operatorem `delete`.

Zwalnianiem pamięci zajmuje się **garbage collector** (GC, odśmiecacz pamięci), czyli część JVM, która działa w trakcie wykonywania programu. Istnieje wiele algorytmów i implementacji GC, ale ich pracę można uprościć do trzech kroków:

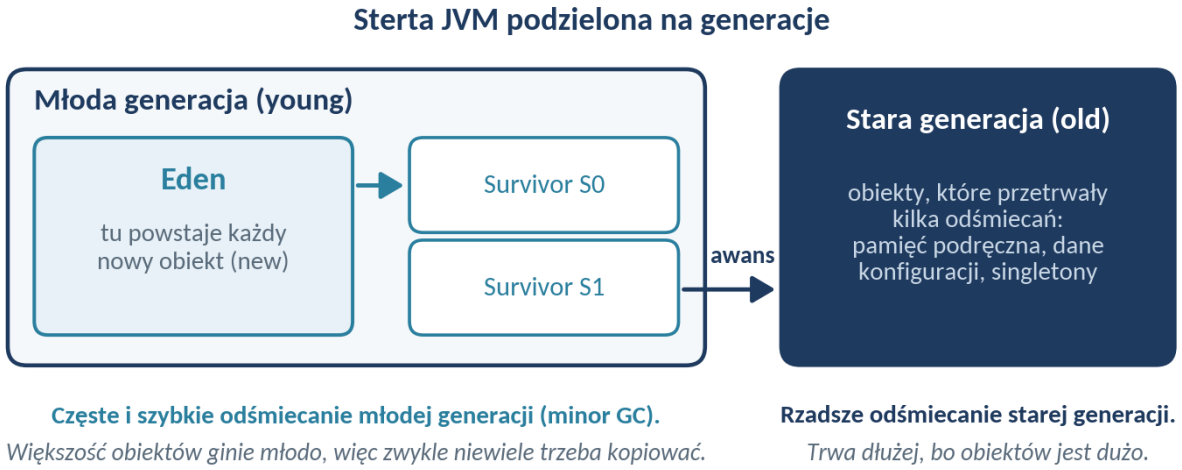
1. **Oznaczanie** (*mark*): GC ustala, których obiektów program już nie używa.
2. **Usuwanie** (*sweep*): pamięć zajmowana przez te obiekty zostaje zwolniona.
3. **Kompaktowanie** (*compact*): używane obiekty są przesuwane obok siebie, dzięki czemu powstaje duży ciągły obszar wolnej pamięci i nie dochodzi do jej fragmentacji.



Rysunek 1. *Trzy kroki pracy garbage collectora*

Aby znaleźć obiekty do usunięcia, GC zaczyna od tak zwanych **korzeni** (*GC roots*). Są to między innymi zmienne lokalne i parametry metod na stosach wszystkich wątków oraz pola statyczne klas. Od korzeni GC przechodzi po kolejnych referencjach i oznacza każdy obiekt, do którego da się w ten sposób dotrzeć. Obiekty, do których program nie ma już żadnej drogi, są uważane za „śmieci", a zajmowana przez nie pamięć jest zwalniana. Dotyczy to także grup obiektów, które wskazują na siebie nawzajem (na przykład dwa obiekty z referencjami do siebie), ale do których nie prowadzi żadna ścieżka od korzeni.

Większość współczesnych kolektorów to **kolektory generacyjne**. Wykorzystują one obserwację, że w programach obiektowych większość obiektów żyje bardzo krótko: tymczasowe napisy, iteratory czy obiekty pomocnicze przestają być potrzebne zaraz po utworzeniu. Sterta jest więc podzielona na **młodą generację**, w której powstają nowe obiekty, i **starą generację**, do której trafiają obiekty, które przetrwały kilka odśmiecań. Młodą generację GC przegląda często i szybko, a starą znacznie rzadziej.



Rysunek 2. *Podział sterty na generacje*

JDK 25 zawiera kilka kolektorów, które różnią się celami: jedne minimalizują przerwy w działaniu programu, inne maksymalizują przepustowość albo zużycie pamięci. Kolektor wybieramy opcją przekazaną do polecenia `java`, na przykład `java -XX:+UseZGC -jar aplikacja.jar`.

Tabela 1. Kolektory dostępne w JDK 25

Kolektor	Opcja JVM	Zastosowanie
G1 (*Garbage-First*)	`-XX:+UseG1GC`	kolektor domyślny; dzieli stertę na regiony i dba o krótkie przerwy przy dobrej przepustowości
ZGC	`-XX:+UseZGC`	przerwy poniżej milisekundy nawet przy stertach rzędu terabajtów; od JDK 24 działa wyłącznie w trybie generacyjnym
Parallel	`-XX:+UseParallelGC`	największa przepustowość kosztem dłuższych przerw; dobry do obliczeń wsadowych
Serial	`-XX:+UseSerialGC`	jednowątkowy, najprostszy; do małych aplikacji i maszyn z jednym procesorem
Shenandoah	`-XX:+UseShenandoahGC`	krótkie przerwy jak w ZGC; dostępny w OpenJDK (np. Temurin), ale nie w Oracle JDK

> **WARTO WIEDZIEĆ**
>
> G1 jest domyślny od JDK 9, ale tylko na maszynach, które JVM uznaje za serwerowe (co najmniej dwa procesory i 1792 MB pamięci). Na słabszym sprzęcie, na przykład w małym kontenerze z jednym procesorem, JVM wybiera kolektor Serial. Kolektor CMS (*Concurrent Mark Sweep*), często opisywany w starszych materiałach, został usunięty w JDK 14 i opcja `-XX:+UseConcMarkSweepGC` nie jest już rozpoznawana.

W większości przypadków programista nie musi wiedzieć, jak dokładnie działa GC ani go konfigurować. Domyślne ustawienia dobrze sprawdzają się w typowych programach. Ta wiedza przydaje się dopiero w aplikacjach o dużym obciążeniu. Wtedy zwykle zaczynamy jednak od innych metod optymalizacji: ograniczenia liczby tworzonych obiektów, doboru rozmiaru sterty opcjami `-Xms` i `-Xmx` oraz obserwacji działania GC, a dopiero potem zmieniamy kolektor.

1.2. Ręczne wywoływanie GC

W programie możemy poprosić maszynę wirtualną o uruchomienie odśmiecania:

```
System.gc();
```

Jest to jednak tylko **sugestia**. Specyfikacja nie gwarantuje, że GC w ogóle się uruchomi, bo ostateczną decyzję podejmuje JVM. Uruchomienie z opcją `-XX:+DisableExplicitGC` sprawia, że wywołania `System.gc()` są całkowicie ignorowane. Z kolei gdy JVM potraktuje prośbę poważnie, zwykle wykonuje pełne odśmiecanie całej sterty, które zatrzymuje program na dłużej niż zwykłe odśmiecanie młodej generacji. Dlatego w kodzie produkcyjnym nie należy wywoływać `System.gc()`. W przykładach na tych zajęciach używamy tej metody wyłącznie po to, żeby pokazać działanie różnych rodzajów referencji.

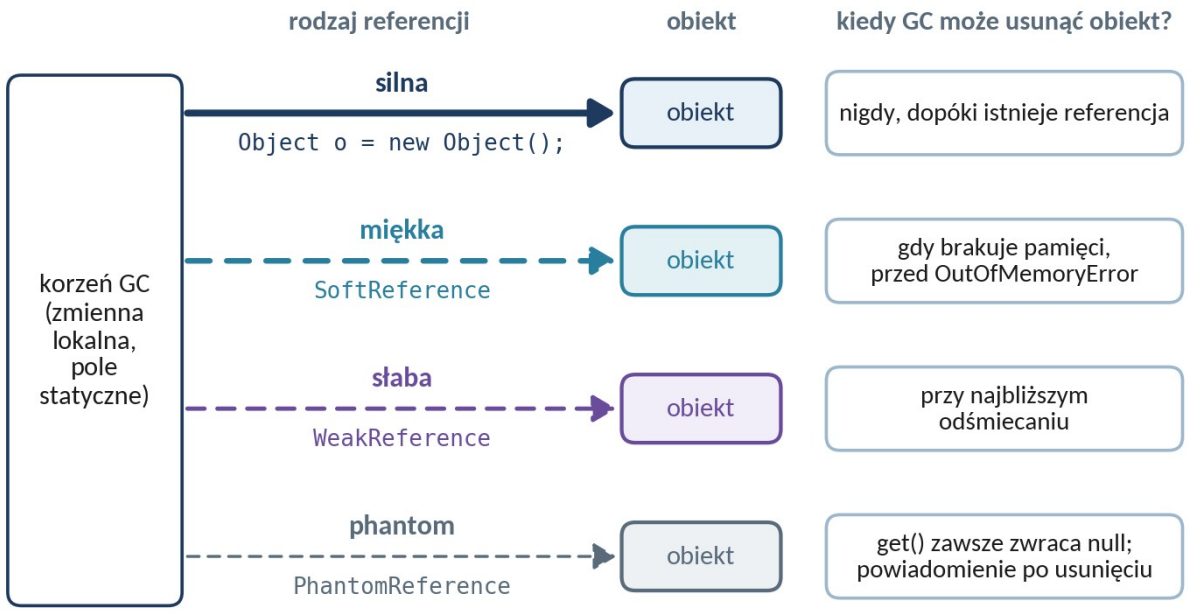
Pracę GC możemy obserwować, uruchamiając program z opcją `-Xlog:gc`. JVM wypisze wtedy, jakiego kolektora używa, oraz informację o każdym odśmiecaniu, wraz z jego przyczyną, zajętością sterty przed i po oraz czasem trwania. Poniżej wynik dla programu `Referencje`, który omówimy w punkcie 2.3:

Terminal Terminal

```
> java -Xlog:gc Referencje
[0.005s][info][gc] Using G1
[0.033s][info][gc] GC(0) Pause Full (System.gc()) 1M->1M(10M) 4.562ms
```

2. Referencje silne, miękkie, słabe i phantom

Referencja, którą zapisujemy w zwykłej zmiennej, jest referencją **silną**. Pakiet `java.lang.ref` udostępnia trzy dodatkowe rodzaje referencji, które nie blokują usunięcia obiektu, ale w różnym stopniu wpływają na to, kiedy GC może go usunąć. Obiekt, do którego chcemy mieć taką referencję, przekazujemy do konstruktora odpowiedniej klasy (`SoftReference`, `WeakReference` lub `PhantomReference`), a dostęp do niego uzyskujemy metodą `get()`.



Rysunek 3. *Rodzaje referencji i to, kiedy GC może usunąć wskazywany obiekt*

Tabela 2. Porównanie rodzajów referencji

Referencja	Kiedy GC usuwa obiekt	Co zwraca `get()`	Typowe zastosowanie
silna	nigdy, dopóki referencja istnieje	nie dotyczy, to zwykła zmienna	wszystkie zwykłe zmienne i pola
miękka	dopiero gdy brakuje pamięci, zawsze przed `OutOfMemoryError`	obiekt albo `null` po usunięciu	pamięć podręczna wrażliwa na ilość pamięci
słaba	przy najbliższym odśmiecaniu	obiekt albo `null` po usunięciu	dane dodatkowe przypisane do obiektów, `WeakHashMap`
phantom	po usunięciu obiektu referencja trafia do kolejki	zawsze `null`	powiadomienie o usunięciu obiektu, sprzątanie zasobów

2.1. Silne referencje

Z tym rodzajem referencji mamy do czynienia w większości przypadków. W Javie **silna referencja** to odwołanie do obiektu, które uniemożliwia GC jego usunięcie. Wszystkie zmienne i pola typów obiektowych są domyślnie silnymi referencjami. Obiekt pozostaje w pamięci tak długo, jak prowadzi do niego ścieżka silnych referencji od któregoś z korzeni. Gdy ostatnia taka referencja zniknie (zmienna wyjdzie z zasięgu albo przypiszemy jej `null`), obiekt staje się kandydatem do usunięcia.

```
Double pi = 3.1415;      // przykład silnej referencji
System.out.println(pi);
pi = null;               // obiekt może zostać usunięty przez GC
```

W powyższym przykładzie zmienna `pi` jest silną referencją do obiektu `Double`. Dopóki wskazuje ten obiekt, GC go nie usunie. Po przypisaniu `null` do obiektu nie prowadzi już żadna referencja, więc przy najbliższym odśmiecaniu zostanie usunięty.

Silne referencje mogą prowadzić do **wycieków pamięci**. W Javie wyciek nie oznacza pamięci „zgubionej" jak w C, tylko obiekty, których program już nie potrzebuje, ale nadal trzyma do nich referencje. Typowym przykładem jest statyczna lista, do której ciągle dodajemy elementy i z której nigdy ich nie usuwamy. Takich obiektów GC nie może usunąć, więc zużycie pamięci rośnie, aż do błędu `OutOfMemoryError`. Jednym ze sposobów na zapanowanie nad pamięcią są miękkie i słabe referencje, które pozwalają GC usunąć obiekt, gdy nie ma już do niego silnych referencji.

2.2. Miękkie referencje

Drugim rodzajem referencji są **miękkie referencje**. Obiekt osiągalny tylko przez miękką referencję GC usuwa dopiero wtedy, gdy zaczyna brakować pamięci. Specyfikacja gwarantuje, że wszystkie takie obiekty zostaną usunięte, zanim JVM rzuci błąd OutOfMemoryError. Miękkie referencje nadają się więc do przechowywania obiektów, które dobrze mieć pod ręką, ale które w razie potrzeby można odtworzyć, na przykład wczytanych z dysku obrazów albo wyników kosztownych obliczeń. Nie nadają się do danych, których utrata byłaby problemem, bo nie mamy wpływu na to, kiedy GC je usunie.

```
String numerIndeksu = new String("85512"); // nowy obiekt, nie literał z puli
SoftReference<String> softReference = new SoftReference<>(numerIndeksu);
System.out.println(softReference.get()); // 85512
```

Aby utworzyć miękką referencję, tworzymy obiekt klasy **SoftReference** i przekazujemy mu obiekt, na który ma wskazywać, tutaj obiekt ze zmiennej `numerIndeksu`. Nawet jeśli potem przypiszemy tej zmiennej `null`, nadal będziemy mieli dostęp do obiektu przez `softReference.get()`, dopóki GC go nie usunie.

Różnica między referencją silną a miękką polega na tym, że obiekt dostępny tylko przez miękką referencję **może** zostać usunięty. GC zrobi to jednak dopiero wtedy, gdy wolnej pamięci zostanie mało, a do tego czasu zachowuje takie obiekty tak długo, jak to możliwe. Jeśli do obiektu prowadzi jednocześnie jakakolwiek silna referencja, miękka referencja nie ma żadnego znaczenia i obiekt nie zostanie usunięty.

Pełny przykład użycia miękkiej referencji:

Main.java Java

```java
import java.lang.ref.SoftReference;

public class Main {
    public static void main(String[] args) {
        Object obj = new Object();
        SoftReference<Object> softRef = new SoftReference<>(obj);

        // usuwamy silną referencję do obiektu
        obj = null;

        // obiekt nadal jest dostępny przez miękką referencję
        Object retrievedObj = softRef.get();
        if (retrievedObj == null) {
            // obiekt został usunięty przez GC
            System.out.println("Obiekt został usunięty");
        } else {
            // obiekt wciąż jest dostępny
            System.out.println("Obiekt wciąż jest dostępny");
        }
    }
}
```

Po przypisaniu zmiennej `obj` wartości `null` obiekt jest nadal dostępny przez `softRef.get()`. Jeśli GC uzna, że brakuje pamięci, i usunie obiekt, metoda `get()` zacznie zwracać `null`. Dlatego wynik `get()` zawsze sprawdzamy instrukcją warunkową, a jeśli chcemy dalej korzystać z obiektu, zapisujemy go w zmiennej lokalnej (tu `retrievedObj`). Ta zmienna jest znowu silną referencją, więc dopóki jej używamy, obiekt na pewno nie zniknie.

2.3. Słabe referencje

Ostatni rodzaj zwykłych referencji to **słabe referencje**, reprezentowane przez klasę **WeakReference** z pakietu `java.lang.ref`. W odróżnieniu od miękkich nie chronią obiektu nawet przy dużej ilości wolnej pamięci. Jeśli do obiektu prowadzą tylko słabe referencje, GC usunie go przy najbliższym odśmiecaniu obejmującym ten obiekt, a wszystkie słabe referencje do niego zaczną zwracać `null`.

```java
String numerIndeksu = new String("85512");
WeakReference<String> weakReference = new WeakReference<>(numerIndeksu);
numerIndeksu = null;
System.out.println(numerIndeksu); // null
numerIndeksu = weakReference.get();
System.out.println(numerIndeksu); // 85512, o ile GC nie zdążył usunąć obiektu
```

Metoda `get()` pozwala odzyskać silną referencję do obiektu, ale tylko dopóki GC go nie usunął. Po usunięciu obiektu nie da się go już przywrócić. Różnicę między miękką a słabą referencją pokazuje poniższy program:

Referencje.java Java

```java
import java.lang.ref.SoftReference;
import java.lang.ref.WeakReference;

public class Referencje {
    public static void main(String[] args) {
        Object miekki = new Object();
        Object slaby = new Object();
        SoftReference<Object> softRef = new SoftReference<>(miekki);
        WeakReference<Object> weakRef = new WeakReference<>(slaby);

        miekki = null; // usuwamy silne referencje
        slaby = null;
        System.gc();   // prosimy o odśmiecanie

        System.out.println("Miękka: " + softRef.get()); // obiekt zwykle nadal jest
        System.out.println("Słaba:  " + weakRef.get()); // zwykle null
    }
}
```

Terminal Terminal

```
> java Referencje
Miękka: java.lang.Object@2b2fa4f7
Słaba:  null
> java -XX:+DisableExplicitGC Referencje
Miękka: java.lang.Object@2b2fa4f7
Słaba:  java.lang.Object@7ad041f3
```

Przy pierwszym uruchomieniu GC usunął obiekt dostępny tylko przez słabą referencję, a obiekt z miękkiej referencji zostawił, bo pamięci było dużo. Przy drugim uruchomieniu wywołanie `System.gc()` zostało zignorowane, więc oba obiekty przetrwały. To dobra ilustracja tego, że `System.gc()` jest tylko prośbą.

Słabe referencje przydają się, gdy chcemy przechowywać dodatkowe informacje o obiektach, ale nie chcemy, żeby samo ich przechowywanie utrzymywało te obiekty w pamięci. Biblioteka standardowa ma do tego gotową mapę **WeakHashMap**, której klucze są słabymi referencjami: gdy klucz przestaje być używany gdziekolwiek indziej w programie, GC usuwa go razem z wpisem w mapie.

2.4. Referencje phantom

Referencje phantom (*phantom references*, dosłownie „widmowe”) nie służą do odczytywania obiektu. Ich metoda `get()` zawsze zwraca `null`. Służą wyłącznie do tego, żeby dowiedzieć się, **kiedy** GC uznał obiekt za nieosiągalny i można posprzątać związane z nim zasoby, na przykład zamknąć plik albo zwolnić pamięć natywną.

Referencję phantom reprezentuje klasa `java.lang.ref.PhantomReference`. Jej konstruktor wymaga dwóch argumentów: obiektu, który chcemy śledzić, oraz kolejki `java.lang.ref.ReferenceQueue`. Gdy GC stwierdzi, że obiekt jest nieosiągalny, dodaje referencję do tej kolejki. Program może z niej odebrać powiadomienie metodą `poll()` (bez czekania) albo `remove()` (z czekaniem).

PhantomReferenceExample.java Java

```java
import java.lang.ref.PhantomReference;
import java.lang.ref.Reference;
import java.lang.ref.ReferenceQueue;

public class PhantomReferenceExample {
    public static void main(String[] args) throws InterruptedException {
        ReferenceQueue<Object> referenceQueue = new ReferenceQueue<>();
        Object dane = new Object();
        PhantomReference<Object> phantomReference =
                new PhantomReference<>(dane, referenceQueue);

        // get() referencji phantom zawsze zwraca null
        System.out.println("Przed GC: " + phantomReference.get());

        dane = null;  // usuwamy jedyną silną referencję
        System.gc();  // prosimy o uruchomienie odśmiecania

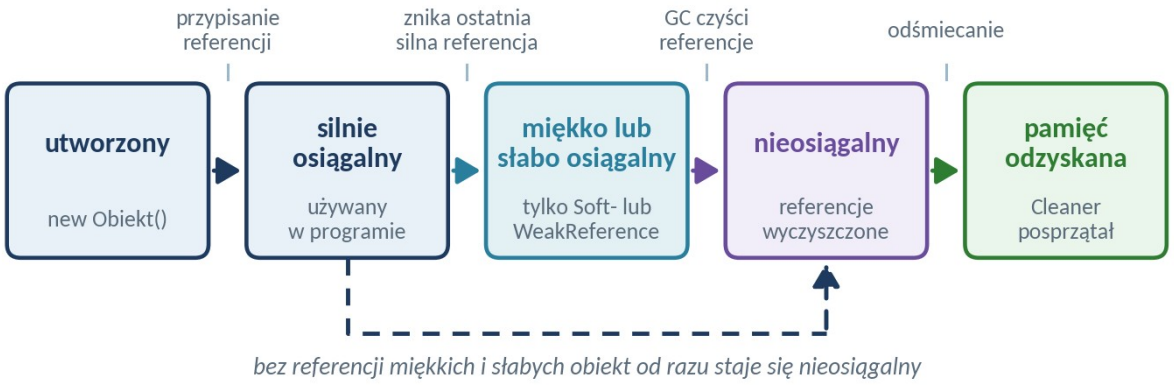
        // czekamy najwyżej sekundę na powiadomienie z kolejki
        Reference<?> ref = referenceQueue.remove(1000);
        System.out.println("Powiadomienie odebrane: " + (ref == phantomReference));
    }
}
```

Wynik Tekst

```
Przed GC: null
Powiadomienie odebrane: true
```

Program tworzy referencję phantom do nowego obiektu, usuwa jedyną silną referencję i prosi o odśmiecanie. Już przed odśmiecaniem `get()` zwraca `null`, bo referencja phantom nigdy nie udostępnia obiektu. Po odśmiecaniu referencja trafia do kolejki, co odczytujemy metodą `remove()`.

> **UWAGA**
>
> Nie śledź w ten sposób literałów napisowych, na przykład `String text = "Hello World"`. Literały trafiają do puli napisów i są stale osiągalne z klasy, która ich używa, więc nigdy nie zostaną usunięte, a referencja nie trafi do kolejki nawet po przypisaniu zmiennej `null`. Z podobnych powodów nie używaj referencji do obiektów klas opakowujących, takich jak `Integer` czy `Double`. Są to klasy oparte na wartości (*value-based*), a na przykład małe liczby `Integer` (od −128 do 127) pochodzą z wewnętrznej pamięci podręcznej i nigdy nie są usuwane. Kompilator Javy 25 zgłasza w takiej sytuacji ostrzeżenie `[identity]`.



Rysunek 4. *Cykl życia obiektu na stercie*

W praktyce rzadko tworzymy referencje phantom samodzielnie. Od Javy 9 biblioteka standardowa udostępnia klasę **Cleaner** z pakietu `java.lang.ref`, która sama korzysta z referencji phantom i własnego wątku. Rejestrujemy w niej obiekt oraz akcję sprzątającą, a `Cleaner` wykona tę akcję, gdy obiekt stanie się nieosiągalny:

Zasob.java Java

```java
import java.lang.ref.Cleaner;

public final class Zasob implements AutoCloseable {
    private static final Cleaner CLEANER = Cleaner.create();

    // stan do posprzątania; klasa statyczna, więc nie trzyma referencji do Zasob
    private static class Stan implements Runnable {
        private final String nazwa;

        Stan(String nazwa) {
            this.nazwa = nazwa;
        }

        @Override
        public void run() {
            System.out.println("Zwalniam zasób " + nazwa);
        }
    }

    private final Cleaner.Cleanable cleanable;

    public Zasob(String nazwa) {
        this.cleanable = CLEANER.register(this, new Stan(nazwa));
    }

    @Override
    public void close() {
        cleanable.clean(); // sprzątanie wykona się najwyżej raz
    }

    public static void main(String[] args) throws InterruptedException {
        try (Zasob z = new Zasob("A")) {
            System.out.println("Używam zasobu A");
        } // close() posprząta od razu
```

```
        new Zasob("B"); // zapomnieliśmy o close()
        System.gc();
        Thread.sleep(500); // dajemy czas wątkowi obiektu Cleaner
    }
}
```

Wynik Tekst

```
Używam zasobu A
Zwalniam zasób A
Zwalniam zasób B
```

Zasób `A` zamykamy jawnie w instrukcji try-with-resources, więc sprzątanie wykonuje się od razu. O zamknięciu zasobu `B` zapomnieliśmy, ale `Cleaner` posprzątał po nim, gdy GC uznał obiekt za nieosiągalny. Akcja sprzątająca nie może trzymać referencji do sprzątanego obiektu (dlatego `Stan` jest statyczną klasą zagnieżdżoną, a nie lambdą korzystającą z `this`), bo wtedy obiekt nigdy nie stałby się nieosiągalny.

> **UWAGA**
> W starszym kodzie można spotkać metodę `finalize()`, którą GC wywoływał przed usunięciem obiektu. Od JDK 18 (JEP 421) finalizacja jest zdeprecjonowana i przeznaczona do usunięcia z języka. Nie wiadomo, kiedy ani czy w ogóle `finalize()` zostanie wywołana, a jej użycie spowalnia GC. Zasoby zwalniamy jawnie metodą `close()` w instrukcji try-with-resources, a `Cleaner` traktujemy tylko jako zabezpieczenie na wypadek, gdyby ktoś zapomniał o `close()`.

3. Pakiety

Pakiety służą do grupowania powiązanych ze sobą klas. Pakiet może zawierać inne pakiety, a cała struktura przypomina katalogi w systemie plików. Pełna nazwa klasy zawiera nazwę pakietu, dlatego pakiety pozwalają uniknąć konfliktów nazw: dwie klasy o tej samej nazwie z różnych pakietów mają różne pełne nazwy. Przykładowo w bibliotece standardowej istnieją zarówno `java.util.Date`, jak i `java.sql.Date`. Pakiety wpływają też na widoczność: składowe bez modyfikatora dostępu (`public`, `protected`, `private`) są dostępne tylko dla klas z tego samego pakietu.

Przynależność klasy do pakietu deklarujemy słowem kluczowym **package** w pierwszej instrukcji pliku:

```
package pl.psk.lab4;
```

Nazwy pakietów piszemy małymi literami i zaczynamy od odwróconej domeny organizacji, na przykład `pl.psk`. Struktura katalogów musi odpowiadać nazwie pakietu, więc klasa z pakietu `pl.psk.lab4` leży w katalogu `pl/psk/lab4`:

Struktura projektu Struktura

```
src/main/java
└── pl
    └── psk
        └── lab4
            ├── AdnotacjeApplication.java
            ├── Main.java
            └── model
                └── Student.java
```

Plik bez deklaracji `package` należy do **pakietu domyślnego** (nienazwanego). Wystarcza on do małych eksperymentów, ale w prawdziwych aplikacjach się go nie używa, bo klas z pakietu domyślnego nie da się zaimportować w klasach z innych pakietów.

Pełną nazwę klasy, razem z pakietem, możemy wypisać metodą `getName()` obiektu klasy:

AdnotacjeApplication.java Java

```java
package pl.psk.lab4;

public class AdnotacjeApplication {

    public static void main(String[] args) {
        System.out.println(AdnotacjeApplication.class.getName());
        // pl.psk.lab4.AdnotacjeApplication
    }
}
```

3.1. Importowanie klas

Klasy z tego samego pakietu widzą się nawzajem bez żadnych dodatkowych instrukcji. Jeśli chcemy użyć klasy z innego pakietu, musimy ją zaimportować instrukcją **import**. Możemy importować pojedyncze klasy albo wszystkie klasy z pakietu, dopisując na końcu nazwy pakietu symbol `*`. Import z gwiazdką nie obejmuje podpakietów: `import java.util.*;` nie importuje klas z `java.util.concurrent`. Klasy z pakietu `java.lang`, takie jak `String`, `Math` czy `System`, są importowane automatycznie.

Drugim sposobem jest podanie pełnej nazwy klasy (razem z pakietem) w miejscu jej użycia. Nie potrzebujemy wtedy instrukcji `import`, ale kod staje się dłuższy. Stosujemy to głównie wtedy, gdy w jednym pliku potrzebujemy dwóch klas o tej samej nazwie.

Istnieje też **import statyczny** (`import static`), który importuje statyczne składowe klasy, czyli jej stałe i metody statyczne. Możemy ich potem używać bez podawania nazwy klasy.

Poniższy kod pokazuje wszystkie te sposoby:

Main.java Java

```java
package pl.psk.lab4; // klasa Main należy do pakietu pl.psk.lab4

import pl.psk.lab4.model.Student;   // import jednej klasy z innego pakietu
import java.util.*;                 // import wszystkich klas z pakietu java.util
import static java.lang.Math.PI;    // import statyczny stałej PI z klasy Math

public class Main {
    public static void main(String[] args) {
        Student student = new Student("Anna");
        List<String> imiona = new ArrayList<>();
        imiona.add(student.getImie());

        // pełna nazwa klasy zamiast importu
        java.time.LocalDate dzisiaj = java.time.LocalDate.now();

        System.out.println(imiona + " " + dzisiaj + " " + PI);
    }
}
```

Klasa `Student` z pakietu `pl.psk.lab4.model` jest zwykłą klasą z polem `imie`, konstruktorem i metodą `getImie()`. Mimo że pakiet `pl.psk.lab4.model` leży „wewnątrz" pakietu `pl.psk.lab4`, z punktu widzenia Javy są to dwa niezależne pakiety, dlatego klasę `Student` trzeba zaimportować.

> **NOWOŚĆ W JAVIE 25**
>
> Java 25 wprowadziła **import modułów** (JEP 511). Instrukcja `import module java.base;` importuje jednym wierszem wszystkie publiczne klasy ze wszystkich pakietów eksportowanych przez moduł `java.base`, między innymi z `java.util`, `java.io`, `java.time` i `java.util.stream`:
>
> **Modul.java** Java
>
> ```java
> import module java.base;
>
> public class Modul {
>     public static void main(String[] args) {
>         List<String> imiona = new ArrayList<>(List.of("Anna", "Jan"));
>         Map<String, Integer> dlugosci = new HashMap<>();
>         for (String imie : imiona) {
>             dlugosci.put(imie, imie.length());
>         }
>         LocalDate dzisiaj = LocalDate.now();
>         System.out.println(dlugosci + " " + dzisiaj);
>     }
> }
> ```
>
> W kompaktowych plikach źródłowych (bez deklaracji klasy) import `java.base` jest dodawany automatycznie. W zwykłych projektach wciąż zalecamy jawne importy pojedynczych klas, bo od razu widać, skąd pochodzi każda z nich.

4. Adnotacje

Adnotacje to metadane, czyli dodatkowe informacje o programie, które nie zmieniają bezpośrednio działania kodu. Można nimi oznaczać klasy, pola, metody, parametry, zmienne lokalne i inne elementy programu. Znak @ informuje kompilator, że ma do czynienia z adnotacją.

Najważniejsze zastosowania adnotacji to:

- dostarczanie informacji kompilatorowi, na przykład o tym, że metoda nadpisuje metodę klasy nadrzędnej,
- dostarczanie informacji narzędziom, które na etapie kompilacji generują kod, pliki konfiguracyjne (XML, JSON) i tak dalej; takie narzędzia nazywamy **procesorami adnotacji**,
- dostarczanie informacji bibliotekom w czasie działania programu; biblioteki Picocli i Spring odczytują adnotacje przez mechanizm refleksji i na tej podstawie konfigurują aplikację.

Przykład adnotacji:

```java
@Deprecated
```

Tabela 3. Najważniejsze wbudowane adnotacje Javy

Adnotacja	Znaczenie
`@Override`	oznacza metodę, która nadpisuje metodę klasy nadrzędnej lub implementuje metodę interfejsu; jeśli takiej metody nie ma, kompilator zgłosi błąd
`@Deprecated`	informuje, że element jest przestarzały i nie powinien być już używany; jego użycie w innej klasie powoduje ostrzeżenie kompilatora
`@SuppressWarnings`	wyłącza wskazane ostrzeżenia kompilatora, na przykład `@SuppressWarnings("unchecked")`
`@FunctionalInterface`	oznacza interfejs funkcyjny; kompilator sprawdza, czy ma dokładnie jedną metodę abstrakcyjną

W poniższym przykładzie metoda `addValues` klasy `Kalkulator` została oznaczona jako przestarzała. Adnotacji `@Deprecated` powinien towarzyszyć komentarz Javadoc ze znacznikiem `@deprecated`, który podpowiada, czego użyć zamiast niej:

Kalkulator.java Java

```java
package pl.psk.lab4;

public class Kalkulator {

    /** @deprecated zamiast tej metody użyj {@link Math#addExact(int, int)} */
    @Deprecated(since = "1.1")
    public static int addValues(int valueA, int valueB) {
        return valueA + valueB;
    }
}
```

AdnotacjeApplication.java Java

```java
package pl.psk.lab4;

public class AdnotacjeApplication {
    public static void main(String[] args) {
        System.out.println(Kalkulator.addValues(1, 2));
    }
}
```

Program skompiluje się i zadziała, ale kompilator ostrzeże nas o użyciu przestarzałej metody. Szczegóły ostrzeżenia zobaczymy po dodaniu opcji `-Xlint:deprecation`:

Terminal Terminal

```
> javac -Xlint:deprecation pl/psk/lab4/*.java
pl/psk/lab4/AdnotacjeApplication.java:5: warning: [deprecation] addValues(int,int)
in Kalkulator has been deprecated
        System.out.println(Kalkulator.addValues(1, 2));
                                      ^

1 warning
```

IntelliJ IDEA pokazuje to samo bez dodatkowych opcji: wywołanie przestarzałej metody jest przekreślone, a ostrzeżenie pojawia się w panelu **Problems**.

WARTO WIEDZIEĆ

Kompilator nie zgłasza ostrzeżenia, gdy przestarzała metoda jest używana w tej samej klasie, w której ją zadeklarowano, dlatego w przykładzie potrzebne są dwie klasy. Jeśli element ma zostać w przyszłości usunięty, oznaczamy go `@Deprecated(forRemoval = true)`. Wtedy ostrzeżenie `[removal]` pojawia się zawsze, nawet bez opcji `-Xlint`.

5. Wprowadzenie do Picocli

Picocli to biblioteka do tworzenia aplikacji wiersza poleceń w Javie przy minimalnej ilości kodu. Opcje i parametry programu opisujemy adnotacjami, a Picocli samo analizuje argumenty, zamienia je na właściwe typy, sprawdza poprawność i wypisuje czytelne komunikaty o błędach. Obsługuje wiele stylów składni wiersza poleceń, w tym POSIX, GNU i MS-DOS. Generuje też konfigurowalne komunikaty pomocy, które mogą wyróżniać ważne elementy kolorami i stylami ANSI.

W tej instrukcji łączymy Picocli ze Spring Boot. Picocli nie ma na liście zależności w kreatorze Spring Initializr, dlatego dopisujemy je ręcznie w pliku `pom.xml`. Biblioteka `picocli-spring-boot-starter` zawiera samo Picocli oraz integrację ze Springiem. Dodatkowo włączamy procesor adnotacji `picocli-codegen`, który już podczas kompilacji sprawdza poprawność adnotacji Picocli (na przykład wykrywa dwie opcje o tej samej nazwie) i generuje pliki konfiguracyjne potrzebne do kompilacji programu do pliku wykonywalnego narzędziem GraalVM:

pom.xml (fragment) XML

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
</parent>

<properties>
    <java.version>25</java.version>
    <maven.compiler.release>25</maven.compiler.release>
    <picocli.version>4.7.7</picocli.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>info.picocli</groupId>
        <artifactId>picocli-spring-boot-starter</artifactId>
        <version>${picocli.version}</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <configuration>
                <annotationProcessorPaths>
```

```
                    <path>
                        <groupId>info.picocli</groupId>
                        <artifactId>picocli-codegen</artifactId>
                        <version>${picocli.version}</version>
                    </path>
                </annotationProcessorPaths>
                <compilerArgs>
                    <arg>-Aproject=${project.groupId}/${project.artifactId}</arg>
                </compilerArgs>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

UWAGA

Od JDK 23 kompilator `javac` nie uruchamia procesorów adnotacji tylko dlatego, że znajdują się na ścieżce klas. Procesor trzeba wskazać jawnie w sekcji `annotationProcessorPaths`, tak jak powyżej. Jeśli w projekcie używasz też Lomboka, jego procesor dopisujesz w tej samej sekcji jako kolejny element `<path>`.

Spring Boot sam nie wie, że argumenty programu ma przekazać do Picocli, dlatego musimy to połączyć ręcznie. Klasa główna aplikacji implementuje dwa interfejsy Spring Boot: **CommandLineRunner**, którego metoda `run()` wykonuje się po uruchomieniu aplikacji i dostaje jej argumenty, oraz **ExitCodeGenerator**, który przekazuje kod wyjścia programu. Konstruktor przyjmuje dwa obiekty, które wstrzykuje Spring:

- `CommandLine.IFactory` to **fabryka** (przykład wzorca projektowego *Fabryka*), której Picocli używa do tworzenia obiektów poleceń; wersja dostarczona przez `picocli-spring-boot-starter` tworzy je jako komponenty Springa, więc do poleceń można wstrzykiwać inne komponenty,
- `MyCommand` to nasza klasa z definicją polecenia, którą napiszemy za chwilę.

Pole `exitCode` przechowuje kod wyjścia zwrócony przez Picocli: 0 oznacza sukces, a inne wartości błąd.

Lab4Application.java Java

```java
package pl.psk.lab4;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.ExitCodeGenerator;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ConfigurableApplicationContext;
import picocli.CommandLine;

@SpringBootApplication
public class Lab4Application implements CommandLineRunner, ExitCodeGenerator {

    private final CommandLine.IFactory factory;
    private final MyCommand myCommand;
    private int exitCode;

    public Lab4Application(CommandLine.IFactory factory, MyCommand myCommand) {
        this.factory = factory;
```

```
        this.myCommand = myCommand;
    }

    @Override
    public void run(String... args) {
        exitCode = new CommandLine(myCommand, factory).execute(args);
    }

    @Override
    public int getExitCode() {
        return exitCode;
    }

    public static void main(String[] args) {
        ConfigurableApplicationContext context =
                SpringApplication.run(Lab4Application.class, args);
        System.exit(SpringApplication.exit(context));
    }
}
```

Metoda `SpringApplication.exit()` zamyka kontekst Springa i zwraca kod wyjścia pobrany z `getExitCode()`, który przekazujemy do `System.exit()`. Bez tego program zawsze kończyłby się kodem 0, nawet po błędzie.

5.1. Dodanie własnego CLI

Pierwszym krokiem jest utworzenie klasy `MyCommand`. Adnotacja `@Component` informuje Springa, że ma utworzyć obiekt tej klasy i wstrzyknąć go do `Lab4Application`. Adnotacja `@CommandLine.Command` informuje Picocli, że klasa opisuje polecenie wiersza poleceń:

```
@CommandLine.Command(name = "StudentSelect", description = "Wybór studenta",
        mixinStandardHelpOptions = true, version = "1.0 dev")
```

Atrybut `name` to nazwa polecenia widoczna w pomocy, a `mixinStandardHelpOptions = true` dodaje standardowe opcje `-h`/`--help` (pomoc) i `-V`/`--version` (wersja z atrybutu `version`).

Kolejnym etapem jest definicja opcji. Adnotację `@CommandLine.Option` umieszczamy nad polem, do którego Picocli wpisze wartość podaną w wierszu poleceń:

```
@CommandLine.Option(names = {"-s", "--student"},
        description = "Typ studenta: S = stacjonarny, N = niestacjonarny")
```

Na koniec implementujemy logikę polecenia. Klasa implementuje interfejs `Callable<Integer>`, a jej metoda `call()` zwraca kod wyjścia, który Picocli przekaże dalej jako wynik `execute()`. Komunikaty formatujemy znacznikami ANSI Picocli w postaci `@|styl tekst|@`. Stała `Ansi.AUTO` włącza kolory tylko wtedy, gdy terminal je obsługuje.

MyCommand.java Java

```
package pl.psk.lab4;

import java.util.concurrent.Callable;
import org.springframework.stereotype.Component;
import picocli.CommandLine;

@Component
@CommandLine.Command(name = "StudentSelect", description = "Wybór studenta",
```

```
        mixinStandardHelpOptions = true, version = "1.0 dev")
public class MyCommand implements Callable<Integer> {

    @CommandLine.Option(names = {"-s", "--student"},
            description = "Typ studenta: S = stacjonarny, N = niestacjonarny")
    private String type = "N";

    @Override
    public Integer call() {
        if (type.equalsIgnoreCase("N")) {
            System.out.println(CommandLine.Help.Ansi.AUTO.string(
                    "@|bold,fg(green) Wybrano studenta niestacjonarnego|@"));
        } else if (type.equalsIgnoreCase("S")) {
            System.out.println(CommandLine.Help.Ansi.AUTO.string(
                    "@|bold,fg(green) Wybrano studenta stacjonarnego|@"));
        } else {
            System.err.println(CommandLine.Help.Ansi.AUTO.string(
                    "@|bold,fg(red) Nieznany typ studenta: " + type + "|@"));
            return 1; // kod wyjścia oznaczający błąd
        }
        return 0; // kod wyjścia oznaczający sukces
    }
}
```

Pole `type` ma wartość domyślną `"N"`, której Picocli użyje, jeśli opcja nie zostanie podana. Aplikację budujemy poleceniem `mvn package` i uruchamiamy z pliku JAR. W poniższych wynikach pominęliśmy baner i komunikaty dziennika Spring Boot:

Terminal Terminal

```
> java -jar target/lab4-1.0-SNAPSHOT.jar -s S
Wybrano studenta stacjonarnego
> java -jar target/lab4-1.0-SNAPSHOT.jar --student x
Nieznany typ studenta: x
> java -jar target/lab4-1.0-SNAPSHOT.jar --help
Usage: StudentSelect [-hV] [-s=<type>]
Wybór studenta
  -h, --help             Show this help message and exit.
  -s, --student=<type>   Typ studenta: S = stacjonarny, N = niestacjonarny
  -V, --version          Print version information and exit.
```

WARTO WIEDZIEĆ

Wcześniej do takich poleceń często dodawano adnotację `@SneakyThrows` z biblioteki Lombok. Pozwala ona rzucić wyjątek kontrolowany z metody, która nie deklaruje go w klauzuli `throws`, na przykład z `run()` interfejsu `Runnable`. Lombok nie ignoruje ani nie opakowuje takiego wyjątku, tylko „oszukuje” kompilator. Na poziomie JVM (pliku `.class`) każdy wyjątek, kontrolowany czy nie, może zostać rzucony niezależnie od klauzuli `throws`, dlatego to działa. Adnotacji tej trzeba używać ostrożnie. W naszym przykładzie nie jest potrzebna, bo metoda `call()` interfejsu `Callable` może rzucać dowolne wyjątki.

Opcję możemy uczynić obowiązkową, dodając atrybut `required = true`:

```
@CommandLine.Option(names = {"-s", "--student"},
        description = "Typ studenta: S = stacjonarny, N = niestacjonarny",
        required = true)
```

Jeśli teraz uruchomimy program bez opcji, Picocli wypisze komunikat o błędzie i pomoc, a program zakończy się kodem 2:

Terminal Terminal

```
> java -jar target/lab4-1.0-SNAPSHOT.jar
Missing required option: '--student=<type>'
Usage: StudentSelect [-hV] -s=<type>
Wybór studenta
  -h, --help             Show this help message and exit.
  -s, --student=<type>   Typ studenta: S = stacjonarny, N = niestacjonarny
  -V, --version          Print version information and exit.
```

WSKAZÓWKA

Picocli działa też bez Springa. Wystarczy wtedy zależność `info.picocli:picocli`, procesor `picocli-codegen` i zwykła metoda `main`. Z klasy `MyCommand` usuwamy adnotację `@Component`:

Main.java Java

```java
package pl.psk.lab4;

import picocli.CommandLine;

public class Main {
    public static void main(String[] args) {
        int exitCode = new CommandLine(new MyCommand()).execute(args);
        System.exit(exitCode);
    }
}
```

Literatura dodatkowa

1. Picocli: Quick Guide, krótki przewodnik po bibliotece Picocli.
2. HotSpot Virtual Machine Garbage Collection Tuning Guide, Java SE 25, oficjalny opis działania i strojenia GC.
3. Available Collectors, opis kolektorów dostępnych w JDK 25.
4. Dokumentacja pakietu java.lang.ref: referencje miękkie, słabe, phantom i klasa `Cleaner`.
5. JEP 421: Deprecate Finalization for Removal, uzasadnienie wycofania metody `finalize()`.
6. Packages i Annotations, rozdziały oficjalnego kursu Javy na dev.java.